

Conservatoire Nationale des Arts et Métiers

Projet NFE204

RethinkDb et AirBase

DIAZ Sebastien N° INE =0G5DRJ1EXW0 N° Siscol =000089827

Table des matières

Introduction	2
Installation de la base documentaire.....	2
Installation sur ubuntu.....	2
Exécution	2
Premier pas et initialisation	2
Administration du serveur	3
Création de la base de données	3
Import des données de la base Airbase	4
Installation du client python	5
Insertion du jeu de données	5
Premiers éléments pour le traitement des données.....	5
Création et suppression de table	5
Les premières fonctions	6
Partitionnement des données initiales	7
Premiers pas avec Map Reduce	10
Les fonctions avancées	10
Liste des 10 plus importantes mesures	11
Comptage des mesures par station.....	11
Comptage des mesures par ville	12
Le nombre de mesures par pays	13
Nombre de mesures du NO2 par année.....	14
Analyse Statistique Spatiale.....	14
Les fonctions avancées	14
Création des points géographiques	15
Recherche des distances max, moyenne et minimal.....	18
Variogramme expérimental	18
Architecture et passage à l'échelle.....	21
RethinkDb en mode Cluster	21
La réplication.....	24
Le partitionnement.....	25
Conclusion.....	28

Introduction

Le sujet du projet pour le cours NFE204 porte sur l'utilisation des connaissances acquises sur les bases documentaires et NOSQL.

Pour la partie « choix des données », la base Airbase spécialisée dans la pollution en Europe est mise en pratique au travers du système NoSQL. Elle comprend près de 146000 mesures et contient des données référencées depuis 1970.

Pour la partie, technologique, la base documentaire choisie est RethinkDB. Le système de requête ReQL est le principal sujet de ce projet. Il est mis en pratique à travers diverses analyses et extractions de données de la base Airbase. Une autre partie plus succincte porte sur l'architecture et le passage à l'échelle.

Nous commencerons notre étude par la mise en place de la base documentaire RethinkDB. Nous continuerons par l'insertion des données brutes de la base Airbase au sein de la base documentaire.

Nous enchaînerons en effectuant les premières requêtes élémentaires. Ensuite nous regarderons des requêtes plus complexes et irons jusqu'à utiliser les fonctions géographiques de la base documentaire, tout en étudiant le contenu sur les mesures de pollution.

Nous finirons par regarder le passage à l'échelle et les techniques utilisées dans le cadre de RethinkDB. Il ne sera pas abordé certains points technologiques portant sur le moteur de RethinkDB ou sur le système d'indexation des données.

L'ensemble des traitements sera effectué avec le [Data Explorer](#) du client Web de RethinkDb.

Installation de la base documentaire

Le premier exercice de ce projet est l'installation du système RethinkDB.

RethinkDb fonctionne sur la plupart des plateformes linux ou OS X.

Le système d'exploitation choisi pour recevoir ce système dans le cadre de ce projet est Ubuntu.

Installation sur ubuntu

Pour installer Rethinkdb, les instructions suivantes extraites de la documentation officielle ont été exécutées.

```
source /etc/lsb-release && echo "deb http://download.rethinkdb.com/apt $DISTRIB_CODENAME main" | sudo
tee /etc/apt/sources.list.d/rethinkdb.list
wget -qO- http://download.rethinkdb.com/apt/pubkey.gpg | sudo apt-key add -
sudo apt-get update
sudo apt-get install rethinkdb
```

Exécution

Pour lancer RethinkDB, il suffit de lancer dans une console :

```
rethinkdb --bind all --cache-size 2048
```

L'option "--bind all" permet d'écouter sur toute les adresses de la machine et d'ouvrir le serveur sur l'extérieur. Par défaut, Rethinkdb écoute sur l'adresse locale de la machine.

L'option "--cache-size" permet de définir la taille du cache qui est un élément important pour les performances du serveur. Chaque contenu de la base RethinkDB est découpé en page de 4 kilo octets pour les documents de plus de 250 octets. Pour les documents inférieurs à 250 octets les données sont sauvegardées directement dans l'index. L'index B-Tree est découpé en page de 4 kilo octets.

Par défaut, le serveur calcul la taille nécessaire au cache par la formule suivante:

Taille du cache = (mémoire disponible - 1024 Méga octets) / (2 * mémoire disponible)

Si la valeur est inférieure à 1224 Méga octets, le serveur choisira une taille de cache de 100 Méga octets.

Premier pas et initialisation

RethinkDb est une base documentaire exposant les données sous forme de flux. Ceci permet de faciliter, la mise en place de système orienté temps réel. Ce principe ne sera pas étudié. De ce fait, toute l'architecture tourne autour du streaming que nous utiliserons tout au long de nos requêtes. Tant qu'il n'y a pas de consommateur, les traitements ne tournent pas et les utilisations mémoire sont moindres.

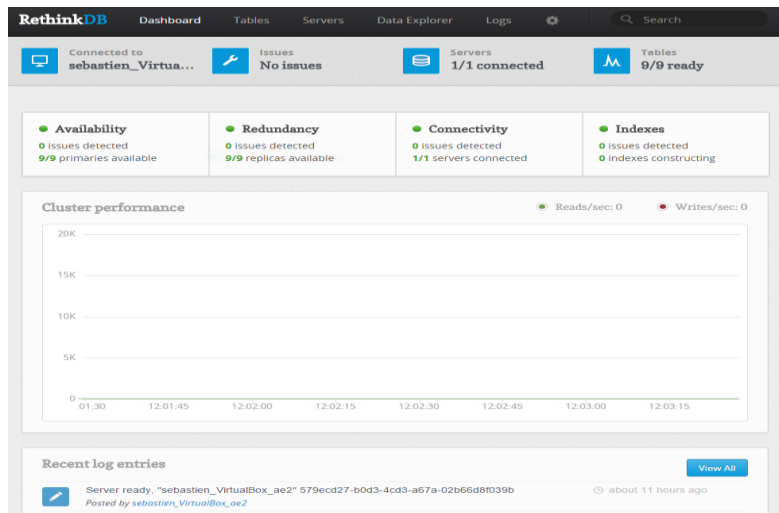
Administration du serveur

Pour accéder au système d'administration web de RethinkDb, il faut lancer un explorateur web et se connecter sur l'url :

<http://localhost:8080>

La page suivante apparaît décrivant les différentes fonctionnalités d'administration.

La première page décrit l'état du serveur



On peut ainsi observer sur une seule page l'activité du serveur, les erreurs, le nombre de serveurs connectés, le nombre d'index, le nombre de table et les performances mesurées.

Il y a d'autres onglets comme la gestion des tables, du serveur, des logs ou de l'explorateur de données.

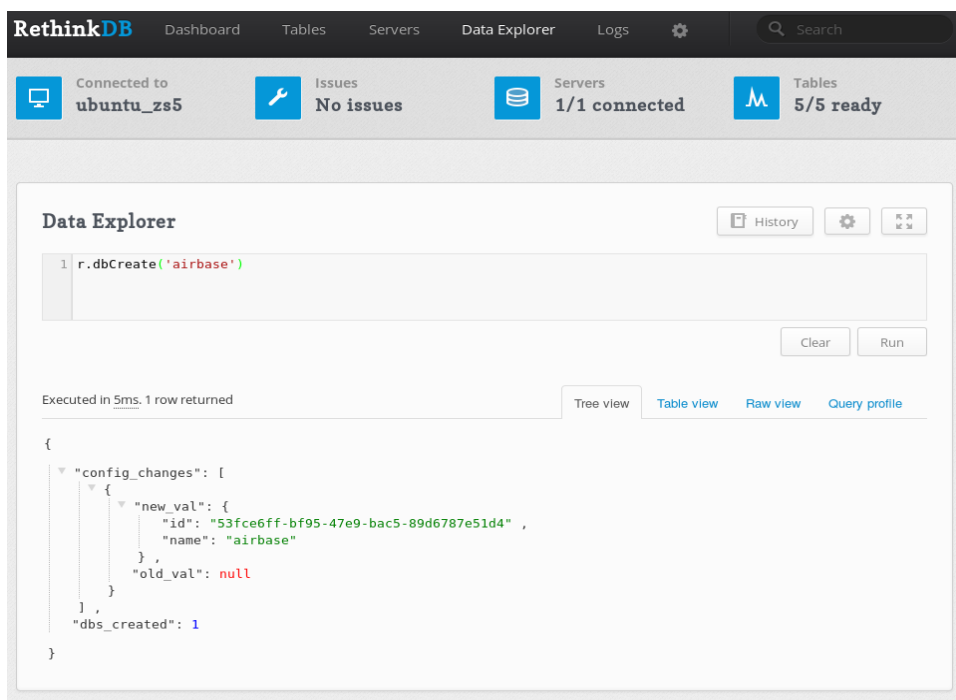
Création de la base de données

La première tâche à effectuer avec un nouvel environnement, est de créer une base qui recevra l'ensemble de nos tables contenant les différents éléments de la base Airbase.

Pour créer notre base, nous devons utiliser l'explorateur de données nommé par l'onglet "data explorer", et lancer la commande suivante :

```
r.dbCreate('airbase')
```

Le résultat est le suivant:



La commande dbCreate permet de créer une base qui contiendra des tables dans lesquels seront stockées les données. Chaque table contient des documents différenciables par une clef primaire

identifiée par défaut par un attribut dans le document nommé "id". Il est important de remarquer que cet identifiant permet la modification et la suppression directement sans passer par un curseur. Lorsqu'un document ne contient pas cet attribut, il est généré automatiquement et signalé en résultat d'insertion.

Pour supprimer la base, il suffira d'entrer: `r.dbDrop('airbase')`

Pour lister, les bases de données, il suffit d'écrire: `r.dbList()`

Avec comme résultat sur notre exemple:

```
[
  "airbase" ,
  "rethinkdb" ,
  "test"
]
```

Dans la liste des bases, on trouve une base nommée "rethinkdb" qui représente les informations du système lui-même.

On peut regarder la liste des tables de cette base, en lançant la commande :

```
r.db('rethinkdb').tableList()
```

Avec comme résultat, la liste des tables du système:

```
[
  "cluster_config" , "current_issues" ,
  "db_config" ,
  "jobs" ,
  "logs" ,
  "server_config" ,
  "server_status" ,
  "stats" ,
  "table_config" ,
  "table_status"
]
```

On y retrouve, l'état du serveur, les erreurs courantes, les jobs en cours, les logs, la configuration du serveur, les statistiques détaillées et la configuration des tables

Import des données de la base Airbase

Les données proviennent de l'url suivante :

<http://www.eea.europa.eu/data-and-maps/data/airbase-the-european-air-quality-database-8>

Les données sont fournies sous forme XML et possèdent de nombreuses informations. Les données seront transformées en JSON pour être intégrées dans le système de gestion No-SQL.

Voici un exemple extrait du jeu de données:

```
<airbase>
<country_name>Andorra</country_name>
<station_name>Engolasters</station_name>
<station_start_date>2006-02-10</station_start_date>
<station_latitude_decimal_degrees>42.516945</station_latitude_decimal_degrees>
<station_longitude_decimal_degrees>1.565014</station_longitude_decimal_degrees>
<station_altitude>91</station_altitude>
<type_of_station>Industrial</type_of_station>
<station_type_of_area>urban</station_type_of_area>
<station_city>KUTINA</station_city>
<population>24597.000</population>
<measurement_configuration component= »Sulphur dioxide (air) - UV fluorescence »>
<component_name>Sulphur dioxide (air)</component_name>
<component_caption>SO2</component_caption>
<measurement_unit>µg/m3</measurement_unit>
<statistics Year= »2006 »>
<statistic_name>annual mean</statistic_name>
<statistic_value>8.477</statistic_value>
```

Nous avons des informations spatiales : les pays, les lieux, les coordonnées géographiques. Il y a aussi des informations contextuelles : les types de stations, les types de lieux, la taille de la population, l'altitude, la date de création de la station. On trouve également des données qualitatives de mesures de pollution avec différentes composantes comme le SO₂, les périodes de mesure, ainsi que les valeurs mesurées (maximum, moyenne par année)...

Le premier travail est de convertir la base initiale, qui est au format XML, au format JSON. Pour cela, l'outil « xml2json.py », qui est disponible à l'adresse <https://github.com/hay/xml2json>, est utilisée.

Le résultat est disponible sur le projet github <https://github.com/StatisticalProject/airbase> dans le répertoire data.

Cette conversion a un certain nombre de spécificités qui sont dus à la particularité de XML.

Par exemple les attributs d'un nœud seront préfixés par le caractère '@'.

Il est difficile de distinguer en XML une séquence ou un seul élément. La résolution en JSON dans ce cas sera soit un élément (quand le nombre de fils est unitaire) soit une séquence (lorsque les fils du nœud sont multiples).

Exemple XML vers JSON

<code><noeud><fils>un</fils></noeud></code>	<code>{"noeud":{"fils":"un"}}</code>
<code><noeud> <fils>un</fils><fils>deux</fils> </noeud></code>	<code>{"noeud": [{"fils":"un"}, {"fils":"deux"}]}</code>

Cela a pour conséquence, dans notre jeu de données, de prendre en considération ces deux types aux travers de procédures spécifiques à chacune d'elle puis de les réunir en une seule séquence.

Installation du client python

Pour utiliser le système d'import fourni avec Rethinkdb, il faut installer le client python.

```
sudo apt-get install python-pip python-dev build-essential
sudo pip install --upgrade pip
sudo pip install --upgrade virtualenv
sudo pip install rethinkdb
```

Insertion du jeu de données

L'importation des données se fera sur la base 'airbase' et sur une table nommée 'origin'.

Voici un exemple d'importation des données sur les pays AD(Andore) et AL(Albanie):

```
rethinkdb import -f data/AD_meta.xml.json --table airbase.origin --force
```

L'option "--force" permet de forcer l'insertion des données quand une table contient déjà des données.

Premiers éléments pour le traitement des données

Toutes les requêtes seront effectuées avec l'onglet « Data Explorer » de l'interface Web de RethinkDB.

Création et suppression de table

Nous allons travailler et découvrir dans ce chapitre les premières fonctions de traitement de l'information. Nous les utiliserons souvent au cours de ce document pour leur grande utilité.

Nous allons réorganiser notre jeu initial de données en plusieurs tables:

- countries : contient toute les informations sur les pays
- stations : contient toutes les informations sur les stations de mesure
- mesures: contient la description de chaque mesure avec leurs statistiques (mean,max,...) par année.

Pour créer une table, il faut utiliser la fonction « tableCreate » qui prend en entrée un nom de table, le nom de l'attribut pour la clef primaire (id par défaut), ainsi que le paramètre "durability" qui permet de gérer la stratégie des changements dans la base

- Deux stratégies pour l'écriture
 - "soft" pas d'attente de vérification d'écriture
 - "hard" attend la vérification de l'écriture sur disque.

Il y a d'autres paramètres qui sont détaillés dans le chapitre sur l'explication du partitionnement et de la réplication.

La création de table est facile, il suffit juste de taper pour chacune des nouvelles tables:

```
r.db('airbase').tableCreate('countries');
r.db('airbase').tableCreate('stations');
r.db('airbase').tableCreate('measures');
```

La commande « tableCreate » contient de nombreuses options permettant le paramétrage de la réplication et du partitionnement. Ces éléments sont détaillés dans le chapitre s’y rapportant.

Pour les supprimer, il suffit de taper

```
r.db('airbase').tableDrop('countries');
r.db('airbase').tableDrop('stations');
r.db('airbase').tableDrop('measures');
```

Pour lister les tables, il suffit de lancer:

```
r.db('airbase').tableList()
```

Avec comme résultat:

```
[
  "countries" ,
  "measures" ,
  "stations" ,
  "origin"
]
```

Les premières fonctions

La fonction map et concatMap

Nous allons utiliser la fonction “map” qui permet depuis une collection de regarder chaque document individuellement et de retourner un nouveau document.

Cette fonction est souvent utilisée avant la phase de réduction et pour transformer les données.

La fonction “map” peut être utilisée depuis une ou plusieurs séquences ou tableaux(array), et générer en sortie une séquence, un tableau ou un “stream”.

Elle prend en argument, une fonction chargé de traiter la transformation individuelle du document.

La procédure “map” gère très mal les documents quand ceux-ci sont des séquences. Le résultat devient une séquence de tableau que l’on préférerait obtenir sous la forme d’une seule séquence. La fonction “concatMap” permet de gérer ce cas de figure.

Un exemple pour comprendre la fonction map:

```
r.expr([{"element1": ["1"]}, {"element1": ["2"]}] ).map(function (doc) {
  return doc('element1');
})
```

Cela donne un résultat peut évident qui est fait de tableau contenu dans un tableau:

```
[ [ "1" ] , [ "2" ] ]
```

Le même exemple mais en utilisant la fonction “concatMap”:

```
r.expr([{"element1": ["1"]}, {"element1": ["2"]}] ).concatMap(function (doc) {
  return doc('element1');
})
```

Le résultat est la concaténation des tableaux en un seul: ["1" , "2"]

La fonction merge

Cette fonction permet de mélanger un document avec un autre.

Exemple : r.expr({element1 : "1"}).merge({element2 : "2"})

Le résultat attendu est le mélange des deux attributs au sein d’un même objet :

```
{"element1": "1" , "element2": "2"}
```

La fonction coerceTo

Pour convertir un élément d’un format à un autre, il faut utiliser la fonction “coerceTo”. Les différents formats de conversion sont: sequence,array,string,number,object,binary.

La fonction typeOf

Pour obtenir le type d’un attribut, il faut utiliser la fonction “typeOf”. Cela permet de traiter les attributs en fonction de leur type “object,array,string,number”.

La fonction without

Pour supprimer d'un document des éléments indésirables, on utilise la fonction "without".

La fonction insert

L'insertion des données dans une table doit se faire par la fonction "insert". Cette fonction prend en paramètre les documents à insérer. D'autres paramètres permettent de changer le comportement comme forcer l'écrasement quand un document est déjà inséré. La détection de doublon est faite par l'attribut "id".

La fonction filter

Une autre fonction importante est celle du filtrage d'une séquence de document. Ici, il n'y a pas de transformations, juste l'élimination du document selon un principe booléen effectué dans une fonction de traitement.

La fonction union

Cette fonction réalise l'union d'une séquence avec une autre séquence.

Les fonctions add,sub,mul,div

Sur les objets de type "number", un certain nombre d'opérations est possible comme l'ajout avec la fonction "add", la suppression avec la fonction "sub", la multiplication avec la fonction "mul" et la division avec la fonction "div". Exemple : `r.expr( 2 ).add(1) -> 3`

La fonction hasFields

Pour savoir si un document possède un ou plusieurs attributs, il faut utiliser "hasFields".

Exemple : `r.expr({ "element1": ["1"], "element2": ["2"] }).hasFields("element1") -> true`

La fonction count

Pour compter le nombre d'éléments d'une séquence ou d'un tableau, il suffit d'appeler la fonction count. Exemple: `r.expr([{"element1": ["1"]}, {"element1": ["2"]}]).count() -> 2`

La fonction distinct

Pour supprimer les duplicatas ou les doublons, il suffit d'utiliser la fonction distinct().

Partitionnement des données initiales

Après description des premières fonctions comme "map", "merge", "coerceTo", "without", "insert" ou "typeOf", nous allons pouvoir commencer nos premiers traitements.

Le traitement des pays

Le premier traitement proposé est l'extraction des pays des documents contenu dans la table "origin". Pour cela, la fonction "map" est utilisé pour transformer le document initial en un document plus léger ne contenant que les informations relatives au pays.

Dans notre cas, il faut supprimer, l'attribut "station" au document "country". La clé primaire est créée à partir du code ISO du pays (FR,ES,...) et est ajouté au document avec la fonction "merge". Puis, elle sera automatiquement utilisée avec l'index par défaut qui se trouve sur l'attribut de nom "id".

Le code est le suivant:

```
//On récupère la description des pays
var countryList= r.db('airbase').table('origin')('airbase')('country') ;
//On convertit le document
var countries= countryList.map(function ( doc) {
//Suppression de la partie station du document et ajout d'un attribut "id"
  return doc.without('station').merge({id: doc('country_iso_code')});
});
//ajout dans la table countries
var table= r.db('airbase').table('countries') ;
table.insert(countries,{conflict:'replace'});
```

Un exemple de document pays :


```
{
  "country_eu_member": "N" ,
  "country_iso_code": "AL" ,
  "country_name": "Albania" ,
  "id": "AL",
  "network": {
    "network_code": "AT001A" ,
    "network_data_supplier": {
      .....
    }
  }
}
```

Le traitement des stations

Nous utilisons la même idée pour les stations. Les précédentes remarques s'appliquent pour cette solution. Cette fois ci, nous supprimons la partie sur les mesures.

```
//On liste chaque station de chaque pays de la base airbase
var stLt= r.db('airbase').table('origin')('airbase')('country')('station');
//On concatene toutes les listes de station
var stations= stLt.concatMap(
function ( doc) {
  //on supprime la partie concernant les mesures
  return doc.without('measurement_configuration');
}).distinct().map(function (doc){ //On ajoute ensuite l'identifiant
  return doc.merge({id:doc('@Id')});
});
//on sauvegarde dans la table stations
var table= r.db('airbase').table('stations') ;
table.insert(stations,{conflict:'replace'});
```

Un exemple de document station :

```
{
  "@Id": "AD0942A:Escaldes-Engordany" , "id": "AD0942A:Escaldes-Engordany" ,
  "network_info":..., "station_european_code": "AD0942A" ,
  "station info": {
    "EMEP_station": "no" ,
    "meteorological_parameter": ... ,
    "population": "22.000" ,
    "sabe_country_code": "AD" , "sabe_unit_code": "AD" , "sabe_unit_name": "Andorra" ,
    "station airbase code": "AD0942A" ,
    "station_altitude": "1080" ,
    "station_city": "ANDORRA LA VELLA" ,
    "station_latitude_decimal_degrees": "42.509724" ,
    "station_latitude_dms": "+042°30'35.00"" ,
    "station_local_code": "942" ,
    "station_longitude_decimal_degrees": "1.539172" ,
    "station_longitude_dms": "+001°32'21.02"" ,
    "station_name": "Escaldes-Engordany" ,
    "station_nuts_level0": "AD" ,
    "station_nuts_level1": "-", ,
    "station_nuts_level2": "-", ,
    "station_nuts_level3": "-", ,
    "station_start_date": "2004-06-17" ,
    "station_type_of_area": "urban" ,
    "type_of_station": "Background"
  }
}
```

Le traitement des mesures

La problématique est ici particulière, car il y a dans le jeu initial de donnée en XML des cas où il n'y a pas de mesure, des cas où il n'y a qu'une seule mesure par station, et des cas où il y a plusieurs mesures. Cela décrit des stations qui produisent des mesures différentes et parfois non présentes dans d'autres stations.

On découpe le traitement en deux parties. Nous commençons par gérer les séquences. Pour cela, nous listons les stations, et pour chaque station nous sélectionnons les éléments de mesure qui sont de type séquentiel ('array').

```
var stations= r.db('airbase').table('origin') ('airbase') ('country') ('station') ;
// Première partie : les tableaux de mesure
var arrayMeasure=stations.map( function (station){
  return station.
    //filtre sur les tableaux et contenant une configuration de mesure
  filter(
    function (doc){
      return doc.hasFields('measurement_configuration').
        and(doc('measurement_configuration').typeof().eq('ARRAY'))
    }
  ).//on concatène les mesures des stations
  concatMap(function (element){
    return element('measurement_configuration').map(
      function (configM){
        //On récupère la mesure et on lui ajoute les informations de station
        //On ajoute un identifiant m »langeant id de stations
        //et le nom de la composante
        return configM.merge(element.without('measurement_configuration')).
          merge({id:element('@Id').add('-').add(configM('component_caption'))});
      }
    );
  });
});
//Seconde partie: les mesures uniques
var objectMeasure=stations.map( function (station){
  //filtre les données avec mesure et qui ne sont pas des tableaux
  return station.filter(
    function (doc){
      return
      doc.hasFields('measurement_configuration').and(doc('measurement_configuration').typeof().eq('OBJECT'))
    }
  )// On transforme les documents
  .map(
    function (element){
      //On récupère la mesure et on lui ajoute les informations de station
      //On ajoute un identifiant mélangeant id de stations
      //et le nom de la composante
      return element('measurement_configuration').
        merge(element.without('measurement_configuration')).
        merge({id:element('@Id').add('-').add(element('measurement_configuration') ('component_caption'))});
    }
  );
});
//On réalise l'union des deux séquences
var mesures=arrayMeasure.union(objectMeasure).concatMap(function (doc){return doc;});
//On sauvegarde dans la table des mesures
var table=r.db('airbase').table('measures');
table.insert(mesures,{conflict:'replace'});
```

On compte le nombre de résultat:

```
r.table('measures').count() -> 26077 mesures
```

On récupère la liste des noms de mesure et on compte les valeurs distinctes.

```
r.table('mesures') ('component_caption').distinct().count() -> 149 type de mesure
```

Un exemple de mesure

```
{
  "@Id": "AD0942A:Escaldes-Engordany" ,
  "@component": "Carbon monoxide (air) - Infrared gas filter correlation" ,
  "component_FWD": "yes" ,
  "component_caption": "CO" ,
  "component_code": "10" ,
  "component_name": "Carbon monoxide (air)" ,
  "id": "AD0942A:Escaldes-Engordany-CO" ,
  "measurement_european_group_code": "100" ,
  "measurement_group_start_date": "2004-06-17" ,
  "measurement_info": {
    "@current_since": "2004-06-17" , "body_or_programme": "EU EoI" ,
    "height_sampling_point": "0" , "length_sampling_line": "0" ,
    "measurement_automatic": "yes" ,
    "measurement_equipment": "Teledyne API 300E gas filter correlation CO analyser" ,
    "measurement_european_code": "100" , "measurement_start_date": "2004-06-17" ,
    "measurement_technique_principle": "Infrared gas filter correlation" ,
    "sampling_time": "0"
  } ,
  "measurement_unit": "mg/m3" , "network_info": ... ,
  "station_european_code": "AD0942A" , "station_info": ...
}
```

Premiers pas avec Map Reduce

Les fonctionnalités MapReduce sont exploités avec les fonctions « map », « group » et « reduce ».

Les fonctions avancées

La fonction group et ungroup

La fonction « group » permet de groupé les documents basé sur le nom d'un attribut. La fonction « ungroup » permet de dégroupier les documents

La fonction reduce

La réduction des documents est effectué par la fonction « reduce ». La fonction « reduce » utilise une fonction qui prend deux documents en entrée pour les mélanger. Le document de gauche est le document regroupé, le document de droite le nouveau document. Au début, il y a deux nouveaux documents, puis ensuite on réduit le document de droite sur le document de gauche. Le résultat se retrouve ensuite sur le document de gauche, et ainsi de suite.

La fonction orderBy

Pour ordonner les documents, on utilise la fonction « orderBy ». Pour choisir, le sens, on utilise les fonctions r.desc() et r.asc().

La fonction limit

La fonction « limit » permet de limité le résultat à un certain nombre.

La fonction eqJoin

La jointure de deux tables est possible et est basé sur la clé primaire de la table à lier.

La fonction match

Pour faire correspondre une information avec une expression régulière, on utilise la fonction match sur l'attribut du document choisi.

La fonction slice

La fonction slice permet de sélectionner une partie d'un binaire.

Liste des 10 plus importantes mesures

Les éléments de réduction permettent un grand nombre de regroupement et d'agrégation des données. Les dix premières mesures sont effectuées en limitant le nombre de résultat avec la fonction « limit ». L'attribut nommant la mesure s'appelle « component_caption ». Le travail consiste donc à grouper les documents par la fonction « group » et cet attribut. Ensuite on retourne la valeur « 1 » à chaque document avec la fonction « map ». Puis on réduit par addition avec la fonction « reduce » toutes ces valeurs de « 1 ». Et enfin, il reste à dégrouper les résultats et à ordonner par la quantité.

```
r.db('airbase').table('measures').group('component_caption').map(
  function (t){
    return 1;
  }).
  reduce(
    function (a,b){
      return a.add(b);
    }
  ).
  ungroup().orderBy(r.desc('reduction')).limit(10)
```

Les dix premières mesures sont :

	group	reduction
1	SO2	2765
2	NO2	2757
3	PM10	2448
4	NOX	2119
5	O3	1831
6	NO	1694
7	CO	1102
8	PM2.5	818
9	C6H6	766
10	Pb	715

Les deux premières mesures de pollution SO2(Dioxyde de soufre) et NO2 (Dioxyde d'Azote) sont très comparable en terme de représentation. Viens ensuite PM10 (Particules en suspensions taille 10) et NOX(Oxyde d'azote).

Une autre solution est possible

Il s'agit d'utiliser le même processus mais avec la fonction « count » qui remplace notre implémentation map reduce. Pour changer d'attribut, on utilise le code de la mesure dans les résultats suivants.

```
r.db('airbase').table('measures').
  group('component_code').
  count().
  ungroup().
  orderBy(r.desc('reduction')).limit(10)
```

Comptage des mesures par station

Pour compter, le nombre de mesure par Station, on commence par joindre la table des mesures avec la table des stations. Puis on récupère la partie de droite nommé 'right' correspondant aux stations. On groupe ensuite par le code de la station. On compte par code puis on dégroupe, trie et récupère les 10 premiers résultats.

```
r.db('airbase').
  //On joint la clé primaire (« id ») de la station
  // à l'attribut @Id du document de la mesure
  table('measures').
  eqJoin("@Id", r.db('airbase').table("stations"))
  ('right').
  group('station_european_code').
  count().ungroup().
  orderBy(r.desc('reduction')).limit(10)
```

Le nom des stations n'est pas très identifiable. On voit que certaines contiennent près d'une centaine de mesure. C'est beaucoup de données, si on ajoute la dimension temporelle.

	group	reduction
1	GB0048R	94
2	NL00934	77
3	GB0036R	76
4	PL0077A	68
5	GB0682A	50
6	GB0777A	50
7	GB0014R	49
8	GB0702A	46
9	LT00002	42
10	FI00351	41

Comptage des mesures par ville

Nos stations possèdent un nom de ville qui va nous permettre de compter le nombre de mesures par ville. A l'instar de la précédente requête, les résultats seront plus identifiables. Nous exécutons la même requête mais en ciblant le nom de la ville («station_city»). Pour cela, nous changeons légèrement la requête pour faire remonter au niveau de la racine du document les informations de la station (« station_info »).

```
r.db('airbase').
  //On joint l'identifiant id de la station
  // a l'@Id du document de la mesure
  table('measures').eqJoin("Id", r.table("stations"))
  ('right').
  map(function (doc){
    //Les informations de station au niveau du document
    return doc('station_info').merge(doc.without('station_info'));
  }).
  group('station_city').count().
  ungroup().orderBy(r.desc('reduction')).limit(10)
```

	group	reduction
1	null	4137
2	LONDON	424
3	BRUSSELS	223
4	WIEN	191
5	DUBLIN	162
6	ANTWERPEN	160
7	PRAHA	145
8	LINZ	140
9	COPENHAGEN	139
10	LIEGE	132

Notre premier constat est de remarquer que le nom de ville des stations n'est pas toujours renseigné. Nous avons plus de 4137 mesures qui sont attaché à aucune ville.

Ce classement nous fait voyager dans toutes l'Europe de Copenhague à Dublin en passant par Prague, Londres et Bruxelles. Ensuite, nous remarquons que Londres possède plus de 424 mesures, ce qui est deux fois plus que la précédente. Londres est une ville très grande en superficie et possède une surface de 1572 km² alors que Bruxelles qui est plus petite ne fait que 32.61 km². Le ratio du nombre des mesures n'est pas comparable avec la surface.

Nous notons aussi la politique et le choix de chaque pays dans l'établissement de système de mesures de pollution. Les villes les plus représentées dans ce classement sont Belges avec Bruxelles, Anvers(Antwerpen) et Liège. Ensuite viens l'Autriche avec Vienne et Linz. Il n'y a pas Paris dans ce classement des dix premières villes. Pourtant c'est une grande ville où la pollution atmosphérique est très présente. On se demande si on peut comparer la pollution de deux villes dans ces conditions. Il faudrait plutôt adresser la métropole comme indicateur. Malheureusement, le projet ne nous laisse pas le temps de regrouper toutes les mesures des villes d'une même métropole.

Le nombre de mesures par pays

Pour mieux comprendre les politiques officielles sur les mesures de pollution, nous changeons de dimension et passons au Pays. Pour cela nous allons extraire les codes pays en utilisant la fonction « slice » pour extraire d'un binaire les deux premiers octets et la fonction « coerceTo » pour faire la transformation string <-> binaire.

```
r.db('airbase').
  //On lit les tables des mesures et des stations
  table('measures').eqJoin("@Id", r.table("stations"))
  ('right').
  //On extrait les deux premières lettres correspondant au code du pays
  map(
    function (po){
      return
      {station_european_code:po('station_european_code').coerceTo('binary').slice(0,2).coerceTo('string')};
    }).
  group('station_european_code').
  count().
  ungroup().
  orderBy(r.desc('reduction')).limit(10);
```

	group	reduction
1	FR	5043
2	GB	3459
3	PL	3025
4	BE	2506
5	AT	1810
6	RO	1485
7	CZ	1221
8	NL	1077
9	PT	709
10	SK	501

Cette vision change notre point de vue. Cette fois ci la France apparait en premier devant la Grande Bretagne. Apparaisse dans ce classement, la Pologne, la Roumanie, le Portugal et la Slovaquie.

Nombre de mesures du NO2 par année

Nous choisissons de travailler uniquement sur la mesure du NO2

```
//On récupère les statistiques uniques
var objectStat=r.db('airbase').table('mesures').
filter(function (pomo){
  return pomo('component_caption').eq('NO2').
and(pomo('statistics').typeOf().ne('ARRAY'));
})
//On récupère tous les éléments
('statistics');
//On récupère des listes de statistiques
var arrayStat=r.db('airbase').table('mesures').
//On filtre sur NO2 et tableau
filter(function (pomo){
  return pomo('component_caption').eq('NO2').
and(pomo('statistics').typeOf().eq('ARRAY'));
}).
//On concatène
concatMap(function (doc){
  return doc('statistics');
});
//On fait l'union
arrayStat.union(objectStat).group('@Year').count().ungroup().orderBy(r.desc('reduction'))
```

	group	reduction
1	2009	1697
2	2008	1689
3	2010	1603
4	2012	1593
5	2011	1571
6	2006	1539
7	2005	1480
8	2007	1468
9	2004	1381
10	2003	1247
11	2002	1136
12	2001	1085
13	2000	1016

Analyse Statistique Spatiale

Les points précédents ont montré les principales caractéristiques de la base NoSql RethinkDB. Mais il y a un autre domaine où RethinkDB excelle : la gestion des données géo référencées. Notre jeu de données étant géo référencé par les coordonnées des stations, de nouvelles possibilités s'offre à nous : recherche par proximité, calcul de distance. Nous allons voir dans ce chapitre comment échantillonner nos données et comment calculer le variogramme d'une mesure. Pour des raisons d'exposé court, nous choisissons de travailler sur la mesure de NO2 et sur l'année 2009.

Les fonctions avancées

La fonction indexCreate

Pour créer un index, on utilise la fonction indexCreate sur la table en précisant le ou les noms des attributs et le type d'index désiré (simple, composé, multiple ou géographique). Dans notre cas, il s'agit d'un index géographique.

La fonction circle, line, point, polygon

Dans rethinkdb, on doit utiliser des formes géographiques pour interagir avec les fonctionnalités géographiques. Les formes géographiques traitées sont les points, les lignes et les polygones. La forme cercle est approximée avec un polygone. L'altitude n'est pas prise en compte. La fonction « point » permet de créer un point géo référencé à partir de coordonnées basé sur la longitude et la latitude. La fonction « circle » prend un point comme centre du cercle et un rayon. Les fonctions « line » et « polygon » complète le lot des formes disponibles.

La fonction getIntersecting

Pour trouver tous les points dans une zone données, on utilise la fonction « getIntersecting » qui permet de faire l'intersection entre une forme précédemment expliquée et les objets géo référencés par l'index précédemment créé.

La fonction distance

Le calcul de la distance entre deux points en coordonnées géographique est effectué avec la fonction « distance ».

Création des points géographiques

Nous débutons nos traitements par l'ajout de point géographique à nos mesures basées sur les coordonnées de nos stations.

```
//on filtre les éléments objets de type NO2 sur l'année 2009
var objectStat=r.db('airbase').table('measures').
  filter(
    function (element){
      var isNO2=element('component_caption').eq('NO2');
      var isArray=element('statistics').typeof().ne('ARRAY');
      return element('@Year').eq('2009').and(isNO2).and(isArray);
    }
  );
//on filtre les éléments objets de type NO2
// puis on récupère toutes les mesure sur l'année 2009
var arrayStat=r.db('airbase').table('measures').
  filter(
    function (doc){
      return doc('component_caption').eq('NO2').and(doc('statistics').typeof().eq('ARRAY'));
    }
  ).
  concatMap(
    function (element){
      return element('statistics').
        filter(
          function (doc){
            return doc('@Year').eq('2009');
          }
        ).
        map(
          function (stat){
            return stat.merge(element.without('statistics')) ;
          }
        );
    }
  );
//On fait l'union et on ajoute le point géoréférencé
var complete=arrayStat.union(objectStat).
  map(
    function (doc){
      var longitude=doc('station_info')('station_longitude_decimal_degrees').coerceTo('number');
      var latitude=doc('station_info')('station_latitude_decimal_degrees').coerceTo('number');
      return doc.merge(
        {point:r.point(longitude,latitude)});
    }
  );
```

Ensuite on récupère toutes les valeurs moyennes des données journalières pour l'année 2009 choisie.

Ceci se fait en deux étapes, d'abord les statistiques multiples par station :

```
//On récupère sur les tableaux, les valeurs moyennes annuels par jour
//On supprime les éléments non nécessaires
var byArray=complete.
  filter(
    function (array){
      return array('statistics_average_group').typeof().eq('ARRAY');
    }).
  //On concatène toutes les valeurs moyennes
  concatMap(
    function (obj){
      return (obj('statistics_average_group').
        filter(
          function (year){
            //On filter sur les données journalières
            return year('@value').eq('day');
          }).
        concatMap(
          function (iniStat){
            ///On gère les listes de mesure
            var resultArray=iniStat('statistic_set')('statistic_result')
              .
              filter(
                function (doc){
                  return doc.typeOf().eq('ARRAY');
                }).
              concatMap(
                function (stat){
                  return stat.filter(
                    function (mean) {
                      return mean('statistic_shortcode').eq('Mean');
                    }
                  );
                }
              );
            //on gère la seul mesure
            var resultObject=iniStat('statistic_set')('statistic_result')
              .
              filter(
                function (doc){
                  return doc.typeOf().ne('ARRAY').and(doc('statistic_shortcode').eq('Mean'));
                }
              );
            return resultArray.union(resultObject);
          }
        )
      ).
      //On supprime les éléments non indispensables
      merge(
        obj.without('statistics_average_group','data_set','measurement_info','station_info','network_info'));
    });
```

On fait de même sur les mesures seules

```
//On récupère sur les objets, les valeurs moyennes annuelles par jour
//On supprime les éléments non nécessaires
var byObject=complete.
  filter(
    function (array){
      return array('statistics_average_group').typeof().ne('ARRAY');
    }).
  concatMap(
    function (objPrincipal){
      var
resultArray=objPrincipal('statistics_average_group')('statistic_set')('statistic_result')
      .
      filter(
        function (doc){
          return doc.typeOf().eq('ARRAY');
        }).
      concatMap(
        function (stat){
          return stat.filter(
            function (mean){
              return mean('statistic_shortcode').eq('Mean');
            });
        } );
      var
resultObject=objPrincipal('statistics_average_group')('statistic_set')('statistic_result').
      filter(
        function (doc){
          return doc.typeOf().ne('ARRAY').and(doc('statistic_shortcode').eq('Mean'));
        });
      var obj= resultArray.union(resultObject);
      return obj.
merge(objPrincipal.without('statistics_average_group','data_set','measurement_info','station_info','network_info')));});
//On fait l'union des tableaux et des objets
var fullComplete=byObject.union(byArray);
```

On crée une nouvelle table contenant nos statistiques.

```
r. db('airbase').tableCreate('statistics') ;
```

Puis on sauvegarde, nos documents retravaillés.

```
r. db('airbase').table('statistics').insert(fullComplete);
```

Exemple de document sauvegardé :

```
{ "@Id": "BA0035A:SARAJEVO ALIPASINA" ,
"@Year": "2009" ,
"@component": "Nitrogen dioxide (air) - Chemiluminescence" ,
"component_FWD": "yes" , "component_caption": "NO2" ,
"component_code": "8" , "component_name": "Nitrogen dioxide (air)" ,
"id": "BA0035A:SARAJEVO ALIPASINA-NO2" ,
"measurement_european_group_code": "100" ,
"measurement_group_start_date": "2008-01-12" ,
"measurement_latest_date_available_in_AIRBASE": "2011-12-31" ,
"measurement_unit": "µg/m3" ,
"point": {
"$req1_type$": "GEOMETRY" , "coordinates": [18.41361 , 43.856388] , "type": "Point"
} , "station_european_code": "BA0035A" , "statistic_name": "annual mean" ,
"statistic_shortcode": "Mean" , "statistic_value": "36.602"
}
```

Recherche des distances maximale, moyenne et minimale

Pour continuer notre analyse, nous avons besoin d'étudier les distances entre stations. Pour des raisons évidentes de complexité de calcul sur un si grand nombre de stations, nous allons effectuer un échantillonnage avec la fonction « sample ». Puis déterminer les distances minimale, maximale et moyenne de cet échantillon.

```
// On échantillonne 200 valeurs
var sample = r.db('airbase').
  table('statistics').sample(200);
// On liste les distances entre chaque station
var list = sample.concatMap(
  function (left) {
    return sample.map(
      function (right) {
        return r.distance(left('point'), right('point'), {unit: 'km'});
      });
  }).
  //on supprime les distances à zéro
  filter(
    function (doc) {
      return doc.ne(0);
    }
  );
r.expr({mean: list.avg(), max: list.max(), min: list.min()});
```

Le résultat est :

```
{
  "max": 13414.312494639706 ,
  "mean": 1212.2436831162736 ,
  "min": 0.5007040257564209
}
```

Variogramme expérimental

Nous allons calculer le variogramme pour la mesure NO2 sur l'année 2009. Le variogramme est une étape importante dans l'étude de régression spatiale comme le krigeage. Le rôle du variogramme est de calculer les variances spatiales sur plusieurs distances permettant une description de la structure spatiale des données. Le variogramme est un calcul qui s'effectue très facilement par un processus MapReduce. Voici la formule du variogramme empirique :

$$\gamma = \frac{1}{2n(h)} * \sum_{h-\delta h < |x-y| < h+\delta h} (z(x) - z(y))^2$$

Les éléments de la formule sont:

- δh est l'intervalle de tolérance de notre distance
- h est la distance
- $2n(h)$ est le nombre de paires de point dans la tolérance de la distance
- $z(x)$ est la mesure au point x

Pour calculer cette fonction selon un modèle MapReduce, il faut effectuer toutes les différences de valeurs portées au carré dans le processus Map. Puis lors du processus de Reduce, il faut faire l'addition de toutes ces valeurs, et enfin diviser, le résultat par deux fois le nombre de mesures.

Nous mettons en place une fonction pour le calcul de chaque élément du variogramme. Ceci permet facilement de calculer tous les points en ne donnant que la distance désirée.

L'unité de distance utilisée est le kilomètre.

Nous choisissons un intervalle de 10% autour de cette distance pour chercher les mesures.

Pour faire une recherche par intersection, nous devons ajouter un index géographique

```
r.db('airbase').table('statistics').indexCreate('point', { geo: true})
```

La recherche de distance est faite avec la fonction `getIntersect` et l'index géographique créé sur l'attribut `point` de chacune de nos mesures. Nous utilisons un cercle de rayon distance plus tolérance. La fonction `getIntersect`, nous retourne les mesures auxquelles nous ajoutons la distance.

Ensuite nous supprimons toutes les mesures dont les distances entre paires de points sont inférieures à la distance moins la tolérance.

Nous obtenons donc toutes les mesures dans l'intervalle distance plus ou moins la tolérance.

Puis nous calculons le variogramme de ces points comme expliqué précédemment.

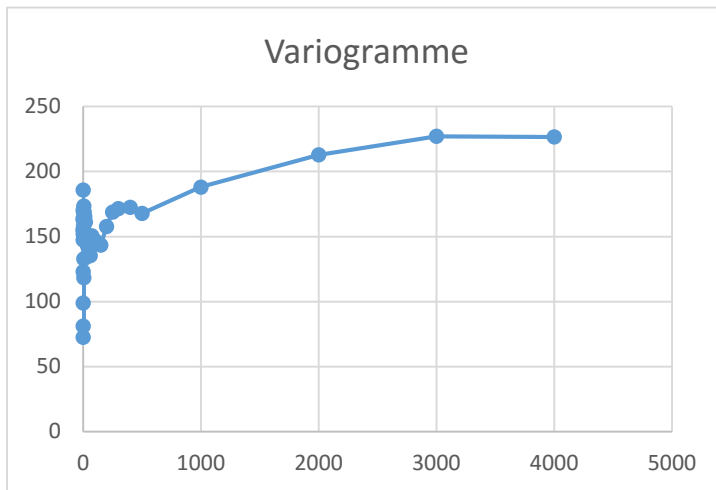
Le point faible de cet algorithme, c'est qu'il ne faut pas dépasser un trop grand rayon sans se retrouver à devoir gérer la table entière.

```
// Fonction calculant le variogramme d'une distance
function variogramme(dist){
  var maxdis=dist*1.1;
  var mindis=dist*0.9;
  //On sélectionne les stations entre les deux distances
  var selection=r.db('airbase').table('statistics').
    concatMap(
      function (addr){
        return r.db('airbase').table('statistics').
          getIntersecting(r.circle(addr('point'), maxdis,{unit:'km'}),{index: 'point'}).
          map(
            function (geo){
              var distance=r.distance(addr('point'),geo('point'),{unit:'km'});
              return geo.without('id').merge({orig:addr,distance:distance});
            }
          );
      }
    ).
    filter(
      function (min) {
        return min('distance').ge(mindis);
      }
    );
  var vario=selection.map(
    function (obj){
      //on soustrait les moyennes
      var subValue=obj('statistic_value').coerceTo('number').
        sub(obj('orig')('statistic_value').coerceTo('number'));
      return {count:1,difference:subValue.mul(subValue)};
    }). // On réduit nos documents en cumulant les différences de valeur au carré
    reduce(
      function (left,right){
        return {
          count:left('count').add(right('count')),
          difference:left('difference').add(right('difference'))};
      }).default(0).
    do(
      function (obj){
        return obj('difference').div(obj('count').mul(2))
      }
    );
  return r.expr({count:selection.count(),distance:dist,value:vario});
}
```

```
variogramme(1.5)
```

Nous exerçons plusieurs mesures pour effectuer un graphique du variogramme.

Nous mesurons aussi le nombre de voisins ayant participé à la mesure.



Le résultat graphique est un variogramme borné, qui montre une augmentation de variance de la mesure de « NO2 » jusqu'à une distance de 1500 km que l'on appelle la portée. La portée s'interprète comme la distance maximale d'auto-correlation spatiale entre deux points et les valeurs de la variable sont considérées indépendantes au-delà. Cela revient à dire que les villes comme Dublin et Hambourg espacées de plus 1100 kilomètres ne souffrent pas d'une cause de proximité quant à leur pollution. On conçoit ici toute la difficulté de la prise de mesure. Il faudrait plutôt parler de différences structurelles entre cités. Nos données sont biaisées car leurs mesures sont massivement

effectuées en centre-ville qui sont souvent pollués par définition. On le voit très bien sur Londres, par exemple, qui dispose du plus grand nombre de points de mesure en Angleterre.

De plus les mesures espacées par un faible kilométrage sont faibles. Le graphique montre ce phénomène qui se caractérise par une variance peu stable sur le faible kilométrage.



La répartition du voisinage permet de se rendre compte que les mesures les plus fournies sont données entre 500 et 2500 km. Ces données ont une représentation spatiale organisée autour des distances entre cités où la pollution est beaucoup plus probable.

Si nous prenons ensemble les caractéristiques expliquées précédemment, l'ensemble des données ne permet d'étudier l'étendue spatiale d'une pollution avec précision.

Seules les structures spatiales peuvent être étudiées grâce notamment aux différents types de polluants caractérisés et par le fait que nous disposons de série chronologique. Des données météo peuvent aussi être facilement ajoutées, pour compléter l'étude.

Architecture et passage à l'échelle

Pour aborder ce chapitre, nous aborderons la notion de cluster au sein de RethinkDB, Puis nous détaillerons la réplication pour finir sur le partitionnement.

La stratégie de Rethinkdb est de maintenir une copie sur chaque serveur de la définition des configurations de partitionnements et de réplication.

RethinkDb en mode Cluster

Introduction et premiers tests

Un serveur rethinkdb possède trois ports de connexions externes. Le premier est le port intra-cluster qui permet à n'importe quel autre serveur de se connecter à la population des serveurs. La démarche pour un nouveau serveur pour entrer dans un cluster est de se connecter sur ce port sur un des serveurs du cluster. Je précise que tous les serveurs ont la capacité d'enregistrer un nouveau serveur sur le cluster. Le chapitre suivant abordera plus précisément ce mécanisme d'enregistrement.

Le deuxième port est le port de connexion au client du serveur RethinkDb.

Le troisième port est le port du serveur Web.

Nous allons démarrer un cluster en démarrant un second serveur et en le connectant au premier.

Lorsque nous avons déjà un serveur démarré sur la même machine les ports ouverts ne sont pas réutilisables rethinkdb possède une option pour pallier à ce problème en proposant de décaler le numéro de ces ports. L'option se nomme « --port-offset » avec le décalage.

Pour se connecter à un cluster, il suffit d'utiliser l'option « --join » avec l'adresse du serveur et son numéro de port intra-cluster.

Il reste à définir un répertoire de données différent que l'on initie avec l'option « --directory ».

On remarquera qu'un cluster est initialisé à chaque démarrage de chaque instance quelque soit le nombre de serveurs lancés.

```
rethinkdb --port-offset 1 --directory rethinkdb_data2 --join localhost:29015
```

Maintenant si nous allons sur la console, nous pouvons voir les deux serveurs lancés.

The screenshot shows the RethinkDB web interface. At the top, there's a navigation bar with links: Dashboard, Tables, Servers, Data Explorer, Logs, and a search bar. Below the navigation bar, there are four status cards: 'Connected to sebastien_Virtua...' with a computer icon, 'Issues No issues' with a wrench icon, 'Servers 2/2 connected' with a database icon, and 'Tables 14/14 ready' with a line graph icon. The main section is titled 'Servers connected to the cluster' and contains a table with two rows of server information.

Server Name	Configuration	Replication Status	Connection Status
sebastien_VirtualBox_ae2	default	14 primaries, 0 secondaries	Connected
sebastien_VirtualBox_trj	default	0 primaries, 0 secondaries	Connected

Le premier serveur lancé contient toutes les tables, alors que le second est vide.

Nous pouvons avoir le statut des serveurs en exécutant la commande suivante dans le Data explorer :

```
r.db('rethinkdb').table('server_status')
```

Le résultat donne les informations sur les serveurs :

```

{
  "connection": {
    "time_connected": Sat Jul 25 2015 15:24:59 GMT+00:00 ,
    "time_disconnected": null
  } ,
  "id": "579ecd27-b0d3-4cd3-a67a-02b66d8f039b" ,
  "name": "sebastien_VirtualBox_ae2" ,
  "network": {
    "canonical_addresses": [
      ..
    ] ,
    "cluster_port": 29015 ,
    "hostname": "sebastien-VirtualBox" ,
    "http_admin_port": 8080 ,
    "reql_port": 28015
  } ,
  "cache_size_mb": 335.171875 ,
  "pid": 3464 ,
  "time_started": Sat Jul 25 2015 15:24:59 GMT+00:00 ,
  "version": "rethinkdb 2.0.4~0trusty (GCC 4.8.2)"
  } ,
  "status": "connected"
}
{

```

```

  "connection": {
    "time_connected": Sat Jul 25 2015 15:26:49 GMT+00:00 ,
    "time_disconnected": null
  } ,
  "id": "f28da52f-3e8f-45a3-bcf7-dd7a98c6f7ba" ,
  "name": "sebastien_VirtualBox_trj" ,
  "network": {
    "canonical_addresses": [
      ..
    ] ,
    "cluster_port": 29016 ,
    "hostname": "sebastien-VirtualBox" ,
    "http_admin_port": 8081 ,
    "reql_port": 28016
  } ,
  "cache_size_mb": 212.60546875 ,
  "pid": 3599 ,
  "time_started": Sat Jul 25 2015 15:26:49 GMT+00:00 ,
  "version": "rethinkdb 2.0.4~0trusty (GCC 4.8.2)"
  } ,
  "status": "connected"
}

```

Initialisation et échanges d'informations au sein du cluster

Nous abordons dans ce chapitre un sujet moins détaillé dans la documentation.

Chaque serveur possède une boîte aux lettres (mailbox) et un répertoire (directory). La boîte aux lettres est utilisée pour échanger des messages entre serveur. Pour éviter des phénomènes d'initialisation des boîtes aux lettres, le répertoire contient l'ensemble des informations nécessaires au cluster.

Les informations du répertoire contiennent les adresses de chaque serveur, le nom des boîtes aux lettres et toutes les adresses des tables et des bases.

Lors d'une initialisation d'un nouveau serveur dans le cluster, celui-ci se connecte au port intra-cluster d'un des serveurs du cluster. Ce dernier lui envoie le répertoire complet ce qui lui permet d'initialiser son dialogue avec les autres éléments du cluster.

Le nouveau serveur instancie de nouvelles connexions aux serveurs du cluster selon un schéma optimisant le nombre de connexions.

L'ensemble des communications est assuré par TCP.

Exemple, nous lançons trois serveurs les uns après les autres. Le premier démarre seul. Le second démarre et se connecte au port intra-cluster du premier. Le troisième démarre en pointant le port intra-cluster du premier serveur.

Le schéma des connexions entre serveur est celui-ci :



La construction des connexions a suivi le schéma suivant. Le premier serveur a démarré et initialisé son répertoire avec lui seul. Le second serveur a démarré et s'est initialisé en copiant le répertoire du premier serveur, s'est ajouté dans le répertoire et a envoyé ce nouveau répertoire au premier serveur qui l'a mis à jour. Le troisième a démarré et a initialisé son répertoire avec le premier serveur et s'est ajouté comme nouveau serveur. Puis il a créé une nouvelle connexion sur le port intra-cluster du second serveur. Et enfin, il a envoyé au premier et au deuxième serveur le nouveau répertoire au travers du système de messagerie par boîte aux lettres.

Pour savoir si un serveur est toujours en vie, les serveurs s'échangent régulièrement des statuts de fonctionnement. Si un des statuts n'a pas répondu, le serveur incriminé est jugé défectueux et déclaré comme tel dans le répertoire.

Ce schéma à l'avantage que lorsqu'un élément s'arrête, le reste des serveurs continue à communiquer. Le système de boîte aux lettres est un processus créé et nommé avec une adresse spécifiquement sur chaque serveur. Cette boîte aux lettres peut être envoyée sur un autre serveur du cluster. Les autres serveurs peuvent utiliser l'adresse pour communiquer avec la boîte aux lettres. Si un serveur envoie un message à une adresse qui n'existe pas ou à une boîte aux lettres défaillante, le message est supprimé et n'est pas renvoyé automatiquement. Les messages convoyés dans les boîtes aux lettres sont de nature différente comme les requêtes utilisateurs, les changements de configuration des tables ou le statut du serveur.

La cohérence des données de configuration du cluster est organisée autour du principe d'un semilattice. Un lattice complet dans le cadre de la distributivité comprend deux opérations. L'opération « meet » qui est la borne inférieure et l'opération « join » qui est la borne supérieure.

Un semi lattice est un semi groupe associatif, commutatif et idempotent, composé d'un ensemble S et d'une opération binaire meet $\wedge$ ou join $\vee$.

Dans le cas de rethinkdb c'est l'opération join qui est utilisée.

Pour $\langle S, \vee \rangle$, on définit les caractéristiques :

- Associatif : $x \vee (y \vee z) = (x \vee y) \vee z$
- Commutatif : $x \vee y = y \vee x$
- Idempotent : $x \vee x = x$

On définit un ordre partiel $\langle S, \leq \rangle$ sur $\langle S, \vee \rangle$.

Pour tout x et y dans S , il existe la relation $x \leq y$ si et seulement si $x \vee y = y$.

La distributivité d'un semilattice est une autre caractéristique.

Pour tout a, b ou x , si $x \leq a \vee b$, alors il existe a' et b' tel que $a' \leq a$, $b' \leq b$ et $x = a' \vee b'$.

Semilattice est très utilisé lorsque des problèmes de grandes échelles, comme les clusters, sont mis en places. Car de nombreux effets, notamment dû à la latence des messages ou aux conflits engendrés par de très gros volume de données, perturbent fortement les fonctionnements des applications.

L'exemple souvent rencontré est de se trouver en face de messages mélangés et désordonnés.

Prenons un exemple simple portant sur trois messages qui sont envoyés dans l'ordre suivant v_1 , v_2 et v_3 . Les trois messages arrivent désordonnés sous cet ordre v_3 , v_1 et v_2 . Nous désirons joindre ces trois messages et nous suivons donc l'algèbre prédéfini.

Réordonnons

$v_3 \vee v_1 \vee v_2 \rightarrow v_3 \vee (v_1 \vee v_2)$: associativité

$v_3 \vee (v_1 \vee v_2) \rightarrow (v_1 \vee v_2) \vee v_3$: commutativité

Joignons

$v_1 \leq v_2 \rightarrow v_1 \vee v_2 = v_2 \rightarrow (v_1 \vee v_2) \vee v_3 \rightarrow v_2 \vee v_3$: cas de v_1 et v_2

$v_2 \leq v_3 \rightarrow v_2 \vee v_3 = v_3$: cas de v_2 et v_3

Le résultat est que le dernier message devient le résultat de la jointure des trois.

Dans RethinkDb, la notion d'ordre passe par un système de versionnement. Chaque donnée de la configuration en possède une. La version la plus importante devient le résultat de la jointure. Si deux versions sont créées simultanément, alors la jointure aura lieu sur l'horloge du serveur ajouté à chacun des messages créés.

La réplication

La réplication nommée « replica » permet de dupliquer une table sur un ou plusieurs serveurs en fonction du nombre de réplique que l'on désire. Nous obtenons par ce principe une sûreté de fonctionnement permettant d'éviter la perte de données.

Les répliques sont de deux sortes : primaire (primary) et secondaire (secondary). Toutes les requêtes en lecture ou en écriture sur une table sont routées sur le réplica primaire.

Lorsque celui-ci est absent, seule la lecture est garantie sur les répliques secondaires. Si le serveur est défectueux et ne peut être remonté, il faut faire une action manuelle sur le serveur pour supprimer le serveur du cluster et remettre en place la réplication comme installée lors du paramétrage de la table.

Il y a une notion de tag qui peut être spécifiée lors du démarrage du serveur par l'option `--server-tag`. On peut en utiliser plus d'un par serveur. Son utilité est de pouvoir découper les répliques par tag.

Exemple une table peut avoir trois répliques sur le tag « us » et deux répliques sur le tag « europe ». Il est possible de choisir dans quel tag sera le réplica primaire avec l'option `primary_replica_tag` lors de la création des répliques ou lors de l'appel à la fonction `reconfigure`.

Les modes de lecture

Il y a deux modes de lecture. Le mode « up to date » (par défaut) qui lit sur le réplica primaire et un mode « out of date » où les requêtes sont exécutées sur le réplica le plus proche.

L'opération est caractérisée par l'utilisation du paramètre `useOutdated` sur la table.

```
Exemple : r.db('airbase').table('countries', {useOutdated: true})
```

Les modes d'écriture

Il y a dans rethinkdb, la possibilité de choisir une des deux stratégies de sauvegarde et de le spécifier pour chaque table ou pour chaque insertion dans une table.

Il y a le mode « soft » qui valide la requête avant même que la requête soit sauvegardée sur les disques. Le serveur se charge d'insérer les données et de faire les répliques dans un mode décalé.

Et il y a le mode « hard » qui est le mode par défaut et qui vérifie que les données sont bien écrites sur le disque et dans le cache. Le mode « hard » fait intervenir trois nouvelles stratégies de sauvegarde dans le cluster :

- **majority** : on attend que la majorité des répliques ait été sauvegardée avec le document, avant de répondre au client. C'est le mode par défaut.
- **single** : il suffit qu'un seul réponde pour répondre au client.
- **complex** : c'est le mode mixte qui découpe en groupe de réplica « single » ou « majority »

Exemple d'insertion des données de pays en mode « soft » :

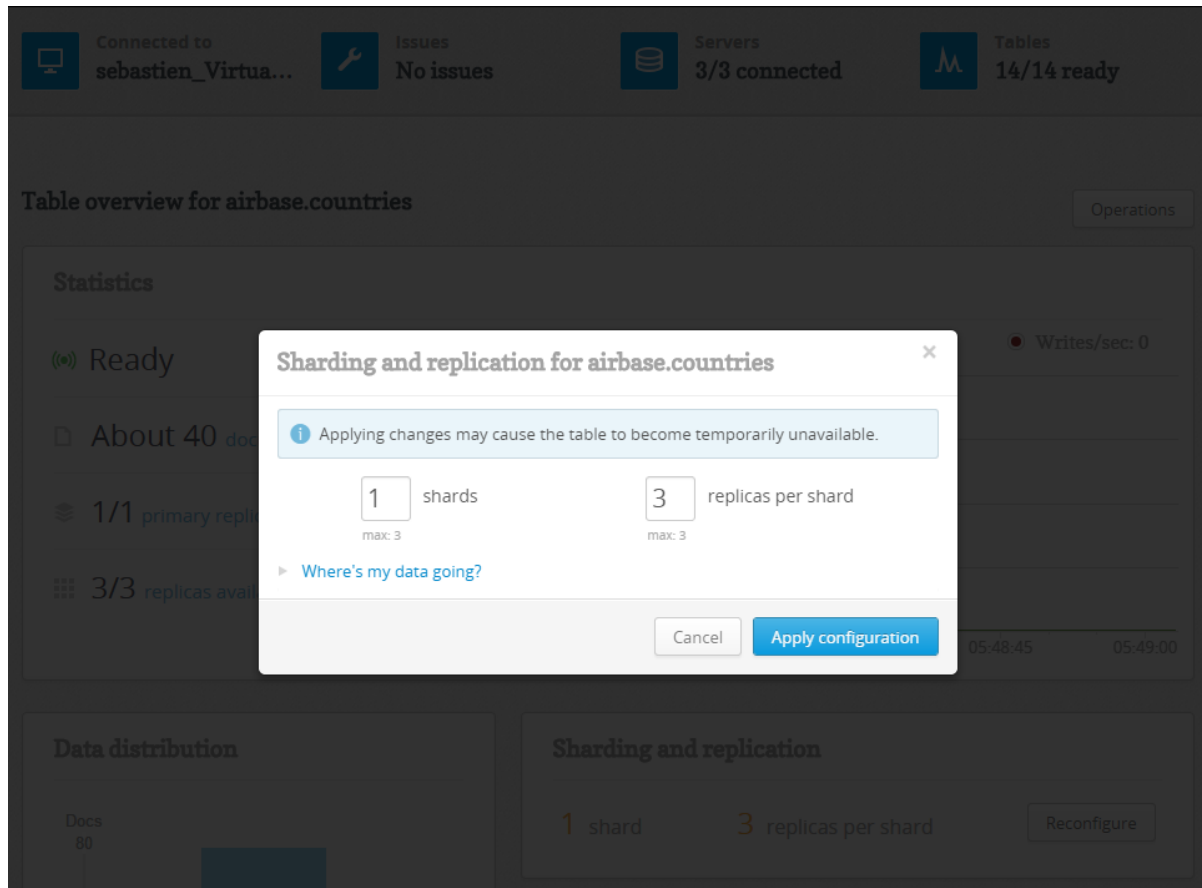
```
r.db("airbase").table("countries").insert(countries,{durability: soft})
```

Pour gérer la stratégie en cluster, on ne peut pas le faire via l'interface web ou par la méthode « reconfigure », il faut changer la table système directement:

```
r.db('rethinkdb').table('table_config').get(
  '31c92680-f70c-4a4b-a49e-b238eb12c023').update(
  {"write_acks": "single"}).run(conn)
```

Création et reconfiguration des réplicas

La configuration de la réplication comme du partitionnement peut être faite par la console web en cliquant sur le bouton reconfigure lorsqu'une table a été sélectionnée.



Il suffit ensuite de sélectionner le nombre de partitions et le nombre de réplicas désirés, puis cliquer sur le bouton « Apply »

La configuration des réplicas peut être faite lors de la création de la table ou lors de l'appel à la fonction « reconfigure ».

Exemple lors de la création d'une table :

```
r.db('airbase').tableCreate('countries', { shards :1, replicas: 3})
```

Avec la fonction « reconfigure » :

```
r.db('airbase').table('countries').reconfigure( {shards :1, replicas :3})
```

Avec plusieurs tags :

```
r.table('countries').reconfigure({shards:1, replicas :{'us':3,  
'europe':2}, primary_replica_tag:'us'})
```

Au moment de la création des répliques, la table n'est utilisable qu'en lecture seule.

Le partitionnement

Le partitionnement permet de couper la taille des bases en plusieurs morceaux appelés « shards » (fragments). Lorsque les bases sont très grandes et qu'elles ne tiennent pas sur un espace disque, il faut alors les découper.

Rethinkdb a choisi un partitionnement par intervalle sur la clef primaire. La structure de routage est sauvegardée dans la configuration du cluster qui est synchronisé sur tous les serveurs.

Lorsqu'une table est partitionnée, le serveur analyse les statistiques de la table pour déterminer la ou les coupures nécessaire(s) à effectuer pour correspondre au nombre de fragments désirés.

L'équilibrage, qui se fait automatiquement, essaye de donner autant de documents à chaque fragment.

La détermination des coupures est faite durant la création des fragments.

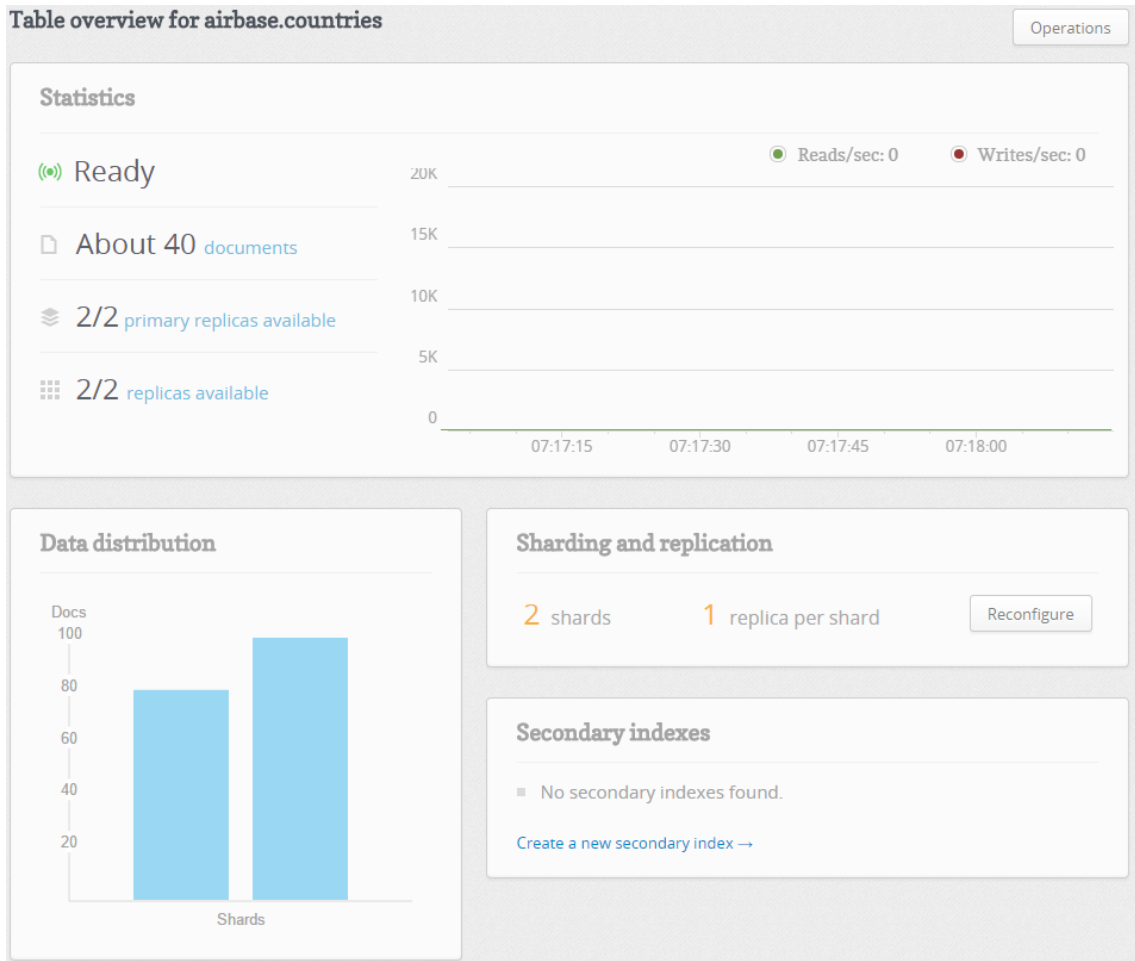
Si les données ne sont pas équilibrées de la même façon dans les prochaines insertions, les fragments risquent de devenir déséquilibrés. A ce moment-là, il faut lancer un nouveau calcul des coupures de fragments en appelant la fonction « rebalance ».

```
r.table('countries').rebalance()
```

Le choix de RethinkDb pour un partitionnement par intervalle a été guidé par la possibilité d'effectuer des requêtes par intervalle. Ce qui n'est pas possible pour un partitionnement par « hashing ». Lors des phases de partitionnement, la table concernée est indisponible.

Création et reconfiguration des « shards »

Comme vu pour la réplication, le nombre de fragments par table est paramétrable par l'application web.



La réplication est aussi configurable par commande lors de la création d'une table ou lors de l'appel à la fonction « reconfigure ».

Exemple lors de la création d'une table :

```
r.db('airbase').tableCreate('countries', { shards :2, replicas: 1})
```

Avec la fonction « reconfigure » :

```
r.db('airbase').table('countries').reconfigure( {shards :2, replicas :1})
```

On peut obtenir le statut du partitionnement en lançant :

```
r.db('rethinkdb').table('table_status')
```

```
{
  "db": "airbase" ,
  "id": "8d8a58a1-fd40-47bb-9e13-f81450559e90" ,
  "name": "countries" ,
  "shards": [
    {
      "primary_replica": "sebastien_VirtualBox_ae2" ,
      "replicas": [
        {
          "server": "sebastien_VirtualBox_ae2" ,
          "state": "ready"
        }
      ]
    } ,
    {
      "primary_replica": "sebastien_VirtualBox_061" ,
      "replicas": [
        {
          "server": "sebastien_VirtualBox_061" ,
          "state": "ready"
        }
      ]
    }
  ] ,
  "status": {
    "all_replicas_ready": true ,
    "ready_for_outdated_reads": true ,
    "ready_for_reads": true ,
    "ready_for_writes": true
  }
}
```

On obtient de la même façon les informations sur la configuration

```
r.db('rethinkdb').table('table_config')
```

```
{
  "db": "airbase" ,
  "durability": "hard" ,
  "id": "8d8a58a1-fd40-47bb-9e13-f81450559e90" ,
  "name": "countries" , "primary_key": "id" ,
  "shards": [
    {
      "primary_replica": "sebastien_VirtualBox_ae2" ,
      "replicas": [
        "sebastien_VirtualBox_ae2"
      ]
    } ,
    {
      "primary_replica": "sebastien_VirtualBox_061" ,
      "replicas": [
        "sebastien_VirtualBox_061"
      ]
    }
  ] ,
  "write_acks": "majority"
}
```

Conclusion

Cette étude nous a mis en exergue plusieurs importants.

Le premier point sur l'analyse des données étudiées. Elles comportent un certain nombre de défaut, de biais de sélection, d'erreurs de mesure ou de spécification qui complique fortement l'analyse statistique que nous pouvions imaginer préalablement. De plus le format XML possède de nombreux défauts une fois transformés en format JSON. Ce ci rend les requêtes compliquées dans notre document.

Néanmoins nous avons pu étudier la structure spatiale des données et nous rendre compte de leurs spécificités liées à des choix territoriaux et politiques. Ceci laisse la possibilité à d'autre analyse, notamment autour des séries chronologiques ou de l'analyse de la variance en fonction des critères disponibles ou rajoutés.

Le deuxième point important est le système documentaire choisi RethinkDB. La première version stable et utilisable a été la version 1.5.0 mis en ligne en Mai 2013. C'est donc une base documentaire récente et qui progresse à chaque version. Sa jeunesse met en évidence un plus faible nombre de fonctionnalités comparé à d'autres acteurs comme MondoDB. Les fonctionnalités présentes aujourd'hui couvrent une grande partie des besoins actuels. La facilité et la rapidité de mise en œuvre d'un cluster, de la réplication ou du partitionnement des données sont très appréciables. Les fonctionnalités très puissantes ont permis une étude efficace des données.

Ce projet a été très formateur pour la compréhension des données étudiées mais aussi la façon de les mettre en place avec la base documentaire RethinkDB.